

EC200U&EG91xU Series QuecOpen(SDK) Device Management Guide

LTE Standard Module Series

Version: 1.1

Date: 2023-09-12

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-02-20	Jensen FANG/ JoJo YAN	Creation of the document
1.0	2021-10-25	Jensen FANG/ JoJo YAN	First official release
1.1	2023-09-12	Kevin WANG	1. Updated the document name based on the unified QuecOpen naming. 2. Added the applicable modules EG912U-GL and EG915U series.

Contents

About the Document.....	3
Contents	4
Table Index.....	5
Figure Index	6
1 Introduction	7
2 API Call Process	8
3 Device Management API	9
3.1. Header File	9
3.2. API Overview.....	9
3.3. API Description.....	10
3.3.1. ql_dev_get_imei.....	10
3.3.1.1. ql_errcode_dev_e.....	10
3.3.2. ql_dev_get_firmware_version.....	12
3.3.3. ql_dev_get_model.....	13
3.3.4. ql_dev_get_sn.....	13
3.3.5. ql_dev_set_modem_fun.....	14
3.3.6. ql_dev_get_modem_fun	14
3.3.7. ql_dev_get_product_id.....	15
3.3.8. ql_dev_get_cpu_uid.....	15
3.3.9. ql_dev_get_temp_value.....	16
3.3.10. ql_dev_cfg_wdt.....	16
3.3.11. ql_dev_feed_wdt.....	17
3.3.12. ql_dev_memory_size_query.....	17
3.3.12.1. ql_memory_heap_state_t.....	17
4 Example	19
4.1. Development Example	19
4.2. Feature Debugging.....	20
5 Appendix References	21

Table Index

Table 1: API Overview	9
Table 2: Related Documents	21
Table 3: Terms and Abbreviations	21

Figure Index

Figure 1: API Call Process	8
Figure 2: Ports in Device Manager	20

1 Introduction

Quectel EC200U and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen[®] solution. QuecOpen[®] is an embedded development platform based on RTOS system. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen[®], see **document [1]**.

This document is applicable to QuecOpen[®] solution based on SDK build environment, and introduces the device management API, its calling process, the related example and feature debugging of EC200U and EG91xU family modules.

2 API Call Process

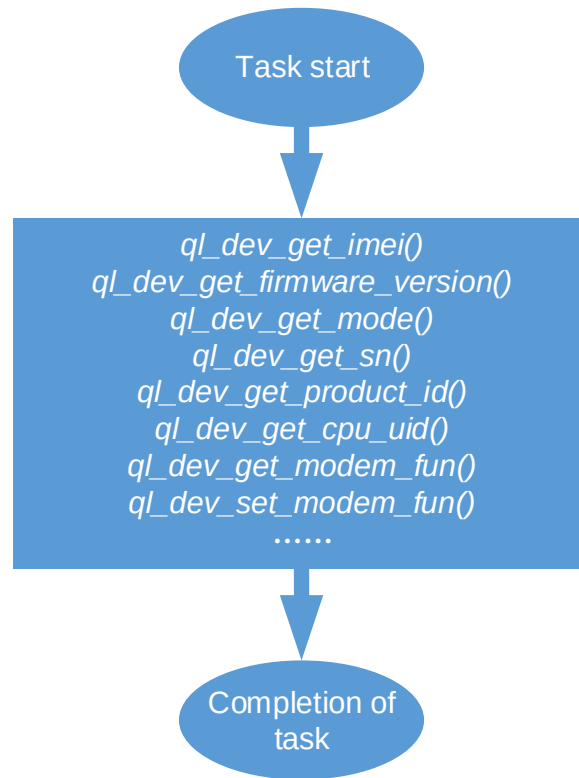


Figure 1: API Call Process

3 Device Management API

3.1. Header File

ql_api_dev.h, the header file of device management API, is in the *components\ql-kernel\inc* directory of the SDK. Unless otherwise specified, all the header files mentioned in this document are in this directory.

3.2. API Overview

Table 1: API Overview

Function	Description
<i>ql_dev_get_imei()</i>	Gets the IMEI of the device.
<i>ql_dev_get_firmware_version()</i>	Gets the firmware version of the device.
<i>ql_dev_get_model()</i>	Gets the device model.
<i>ql_dev_get_sn()</i>	Gets the device serial number.
<i>ql_dev_set_modem_fun()</i>	Sets the modem function of the device.
<i>ql_dev_get_modem_fun()</i>	Gets the current modem function of the device.
<i>ql_dev_get_product_id()</i>	Gets the manufacturer ID of the device.
<i>ql_dev_get_cpu_uid()</i>	Gets the UID of CPU.
<i>ql_dev_get_temp_value()</i>	Gets the chip temperature.
<i>ql_dev_cfg_wdt()</i>	Configures the watchdog timer switch.
<i>ql_dev_feed_wdt()</i>	Feeds the system watchdog (clear the timer).
<i>ql_dev_memory_size_query()</i>	Queries the heap state.

3.3. API Description

3.3.1. ql_dev_get_imei

This function gets the IMEI of the device.

- **Prototype**

```
ql_errcode_dev_e ql_dev_get_imei(char *p_imei, size_t imei_len, uint8_t nSim)
```

- **Parameter**

p_imei:

[Out] IMEI of the device.

imei_len:

[In] Length of the IMEI. Unit: byte.

nSim:

[In] The (U)SIM card used.

0 (U)SIM card 1

1 (U)SIM card 2

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.1.1. ql_errcode_dev_e

The enumeration of function execution error codes:

```
typedef enum
{
    QL_DEV_SUCCESS = QL_SUCCESS,

    QL_DEV_EXECUTE_ERR                                = 1|QL_DEV_ERRCODE_BASE,
    QL_DEV_MEM_ADDR_NULL_ERR,
    QL_DEV_INVALID_PARAM_ERR,
    QL_DEV_BUSY_ERR,
    QL_DEV_SEMAPHORE_CREATE_ERR,
    QL_DEV_SEMAPHORE_TIMEOUT_ERR,

    QL_DEV_CFW_CFUN_GET_ERR                            = 15|QL_DEV_ERRCODE_BASE,
```

```

QL_DEV_CFW_CFUN_SET_CURR_COMM_FLAG_ERR = 18|QL_DEV_ERRCODE_BASE,
QL_DEV_CFW_CFUN_SET_COMM_ERR,
QL_DEV_CFW_CFUN_SET_COMM_RSP_ERR,
QL_DEV_CFW_CFUN_RESET_BUSY                = 25|QL_DEV_ERRCODE_BASE,
QL_DEV_CFW_CFUN_RESET_CFW_CTRL_ERR,
QL_DEV_CFW_CFUN_RESET_CFW_CTRL_RSP_ERR,

QL_DEV_IMEI_GET_ERR                        = 33|QL_DEV_ERRCODE_BASE,

QL_DEV_SN_GET_ERR                         = 36|QL_DEV_ERRCODE_BASE,

QL_DEV_UID_READ_ERR                      = 39|QL_DEV_ERRCODE_BASE,

QL_DEV_TEMP_GET_ERR                     = 50|QL_DEV_ERRCODE_BASE,

QL_DEV_WDT_CFG_ERR                      = 53|QL_DEV_ERRCODE_BASE,

QL_DEV_HEAP_QUERY_ERR                   = 56|QL_DEV_ERRCODE_BASE,

QL_DEV_AUTHCODE_READ_ERR                = 90|QL_DEV_ERRCODE_BASE,

QL_DEV_READ_WIFI_MAC_ERR                = 120|QL_DEV_ERRCODE_BASE,

QL_DEV_AUTHCODE_ADDR_NULL_ERR,
} ql_errcode_dev_e

```

● Member

Member	Description
<i>QL_DEV_SUCCESS</i>	Successful execution.
<i>QL_DEV_EXECUTE_ERR</i>	Failed execution.
<i>QL_DEV_MEM_ADDR_NULL_ERR</i>	Pointer NULL error.
<i>QL_DEV_INVALID_PARAM_ERR</i>	Parameter error.
<i>QL_DEV_BUSY_ERR</i>	The device is busy. Operation failed.
<i>QL_DEV_SEMAPHORE_CREATE_ERR</i>	Failed to create semaphore.
<i>QL_DEV_SEMAPHORE_TIMEOUT_ERR</i>	Semaphore times out.
<i>QL_DEV_CFW_CFUN_GET_ERR</i>	Failed to get the current functional mode.

QL_DEV_CFW_CFUN_SET_CURR_COMM_FLAG_ERR	Setting of functional mode is currently not supported.
QL_DEV_CFW_CFUN_SET_COMM_ERR	Failed to set the functional mode.
QL_DEV_CFW_CFUN_SET_COMM_RSP_ERR	Abnormal response of functional mode setting.
QL_DEV_CFW_CFUN_RESET_BUSY	Shutdown is busy and the previous shutdown process is in progress.
QL_DEV_CFW_CFUN_RESET_CFW_CTRL_ERR	Failed to close the protocol stack.
QL_DEV_CFW_CFUN_RESET_CFW_CTRL_RSP_ERR	Abnormal response of closing the protocol stack.
QL_DEV_IMEI_GET_ERR	Failed to get IMEI.
QL_DEV_SN_GET_ERR	Failed to get the device serial number.
QL_DEV_UID_READ_ERR	Failed to get UID.
QL_DEV_TEMP_GET_ERR	Failed to get chip temperature.
QL_DEV_WDT_CFG_ERR	Failed to configure watchdog timer switch.
QL_DEV_HEAP_QUERY_ERR	Failed to query the heap state.
QL_DEV_AUTHCODE_READ_ERR	Failed to read authorization code of camera decoder library.
QL_DEV_READ_WIFI_MAC_ERR	Failed to read Wi-Fi MAC address.
QL_DEV_AUTHCODE_ADDR_NULL_ERR	Authorization code address of the camera decoder library is NULL.

3.3.2. ql_dev_get_firmware_version

This function gets the firmware version of the device.

● Prototype

```
ql_errcode_dev_e ql_dev_get_firmware_version(char *p_version, size_t version_len)
```

● Parameter

p_version:

[Out] Firmware version of the device.

version_len:

[In] Length of the version number. Unit: byte.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.3. **ql_dev_get_model**

This function gets the device model.

- **Prototype**

```
ql_errcode_dev_e ql_dev_get_model(char *p_model, size_t model_len)
```

- **Parameter**

p_model:

[Out] Device model.

model_len:

[In] Length of the device model. Unit: byte.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.4. **ql_dev_get_sn**

This function gets the device serial number.

- **Prototype**

```
ql_errcode_dev_e ql_dev_get_sn(char *p_sn, size_t sn_len, uint8_t nSim)
```

- **Parameter**

p_sn:

[Out] Device serial number.

sn_len:

[In] Length of the device serial number. Unit: byte.

nSim:

[In] The (U)SIM card used.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.5. ql_dev_set_modem_fun

This function sets the modem function of the device.

- **Prototype**

```
ql_errcode_dev_e ql_dev_set_modem_fun(uint8_t at_dst_fun, uint8_t rst, uint8_t nSim)
```

- **Parameter**

at_dst_fun:

[In] Modem functions to be set. It can be set to 0, 1 or 4.

- 0 Minimum functionality
- 1 Full functionality
- 4 Airplane mode

rst:

[In] Indicates whether to reboot the modem before setting the modem function.

- 0 Do not reboot
- 1 Reboot

nSim:

[In] The (U)SIM card used.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.6. ql_dev_get_modem_fun

This function gets the current modem function of the device.

● Prototype

```
ql_errcode_dev_e ql_dev_get_modem_fun(uint8_t *p_function, uint8_t nSim)
```

● Parameter

p_function:

[Out] Current modem function of the device. It can be 0, 1 or 4.

- 0 Minimum functionality
- 1 Full functionality
- 4 Airplane mode

nSim:

[In] The (U)SIM card used.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.7. ql_dev_get_product_id

This function gets the manufacturer ID of the device.

● Prototype

```
ql_errcode_dev_e ql_dev_get_product_id(char *p_product_id, size_t product_id_len)
```

● Parameter

p_product_id:

[Out] Manufacturer ID of the device.

product_id_len:

[In] Length of the manufacturer ID of the device. Unit: byte.

● Return Value

See **Chapter 3.3.1.1** for details.

3.3.8. ql_dev_get_cpu_uid

This function gets the UID of CPU.

- **Prototype**

```
ql_errcode_dev_e ql_dev_get_cpu_uid(unsigned long long *p_chip_id)
```

- **Parameter**

p_chip_id:

[Out] UID of the CPU.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.9. ql_dev_get_temp_value

This function gets the chip temperature.

- **Prototype**

```
ql_errcode_dev_e ql_dev_get_temp_value(int32_t *ql_temp)
```

- **Parameter**

ql_temp:

[Out] The value of chip temperature in Celsius.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.10. ql_dev_cfg_wdt

This function configures the watchdog timer switch.

- **Prototype**

```
ql_errcode_dev_e ql_dev_cfg_wdt(uint8_t opt)
```

- **Parameter**

opt:

[In] Switch of the watchdog timer.

- 0 Turn off
- 1 Turn on

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.11. ql_dev_feed_wdt

This function feeds the system watchdog (clears the timer).

- **Prototype**

```
ql_errcode_dev_e ql_dev_feed_wdt(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.12. ql_dev_memory_size_query

This function queries the heap state.

- **Prototype**

```
ql_errcode_dev_e ql_dev_memory_size_query(ql_memory_heap_state_t *ql_heap_state)
```

- **Parameter**

ql_heap_state:

[Out] Heap state. See **Chapter 3.3.12.1** for details.

- **Return Value**

See **Chapter 3.3.1.1** for details.

3.3.12.1. ql_memory_heap_state_t

The structure of heap state:

```
typedef struct
{
    uint32_t total_size;
```

```
uint32_t avail_size;
} ql_memory_heap_state_t
```

● Parameter

Type	Parameter	Description
uint32_t	<i>total_size</i>	Total heap memory. Unit: byte.
uint32_t	<i>avail_size</i>	Available heap memory (which may be larger than the actual value). Unit: byte.

4 Example

This chapter mainly introduces how to use the above API to develop and debug the device management in application layer.

4.1. Development Example

Example file of the device management API *ql_dev_demo.c*, provided in the SDK codes of EC200U and EG91xU family modules, is in the *components\ql-application\dev* directory. The example file contains how to get IMEI of the module, the device serial number, and the device firmware version, among others. The entry function is *ql_dev_app_init()* as below.

```
void ql_dev_app_init(void)
{
    QIosStatus err = QL_OSI_SUCCESS;
    ql_task_t dev_task = NULL;

    err = ql_rtos_task_create(&dev_task, QL_DEV_TASK_STACK_SIZE, QL_DEV_TASK_PRIO, "QDEVDEMO", ql_dev_demo_thread, NULL, QL_DEV_TASK_EVENT_CNT);
    if (err != QL_OSI_SUCCESS)
    {
        QL_DEV_LOG("dev demo task created failed");
    }
}
```

As shown in the figure below, the above example has not been enabled by default in the *ql_dev_demo_thread()*. Uncomment the codes when you run a test.

```
static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);

    /*Caution: If the macro of secure boot and the function are opened, down
    .....the secret key cannot be changed forever*/....
    #ifdef QL_APP_FEATURE_SECURE_BOOT
        //ql_dev_enable_secure_boot();
    #endif

    #if 0
        ql_gpio_app_init();
        ql_gpioint_app_init();
    #endif

    #ifdef QL_APP_FEATURE_LEDCFG
        //ql_ledcfg_app_init();
    #endif

    #ifdef QL_APP_FEATURE_AUDIO
        //ql_audio_app_init();
    #endif
    #ifdef QL_APP_FEATURE_HEADSET_DET
        //ql_headset_det_app_init();
    #endif
    #endif

    //ql_dev_app_init();
    //ql_adc_app_init();
    ....
    #ifdef QL_APP_FEATURE_UART
        //ql_uart_app_init();
    #endif
}
```

4.2. Feature Debugging

You can debug the device management feature of EC200U and EG91xU family modules through LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to the PC with a USB cable. The USB AP Log Port mainly displays the system debugging information. You can view information of the example through USB AP Log Port after capturing the log with *cooltools*. For details about log capture, see [document \[2\]](#).

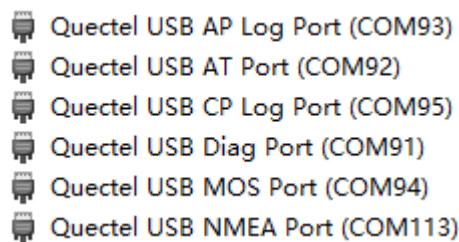


Figure 2: Ports in Device Manager

ql_dev_app_init() is called automatically after the module is turned on. The IMEI, device serial number, device firmware version and other information of the module can be viewed through the log.

The debugging information displayed on the AP Log Port is shown in the figure below:

[ql_DEV][ql_dev_demo_thread, 48]	IMEI: 352273017386340
[ql_DEV][ql_dev_demo_thread, 51]	SN: 98612453164038
[ql_DEV][ql_dev_demo_thread, 54]	Product: Quectel
[ql_DEV][ql_dev_demo_thread, 57]	Model: EC200U
[ql_DEV][ql_dev_demo_thread, 60]	FW: EC200UCNAAR02A01M08_OCPU_BETA210512095844
[ql_DEV][ql_dev_demo_thread, 63]	subVer: V01

5 Appendix References

Table 2: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_CSDK_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
(U)SIM	(Universal) Subscriber Identity Module
AP	Access Point
API	Application Programming Interface
App	Application
CPU	Central Processing Unit
EVB	Evaluation Board
ID	Identifier
IMEI	International Mobile Equipment Identity
LTE	Long-Term Evolution
MAC	Medium Access Control
PC	Personal Computer
RTOS	Real-Time Operating System
SDK	Software Development Kit

UART	Universal Asynchronous Receiver/Transmitter
UID	Unique ID
USB	Universal Serial Bus
Wi-Fi	Wireless Fidelity
